Examen POO 2019

Subiectul 1

Descrieți modurile în care se realizează binding-ul în Java (static, dinamic). Scrieți secvențe de cod Java pentru a exemplifica modul de definire și utilizarea acestuia. Explicați în 2-3 fraze caracteristicile pe care dorit să le evidențiați.

Binding-ul este procesul prin care se stabilește ce metodă sau variabilă va fi apelată.

Static binding-ul se realizează la compilare. Este utilizat de metode și variabile private, final sau static. Metodele supraîncărcate prezintă static binding. Variabilele membru prezintă static binding, deoarece Java nu permite comportament polimorfic pentru variabilele membru.

```
public class Animal
    {   public void write()
            { System.out.println("Animal"); }
    }
public class Dog extends Animal
    {   public static void write()
            { System.out.println("Dog"); }
    }
```

```
public class StaticBinding
{   public static void main (String args[])
    {
        Animal dog = new Dog();
        dog.write();

        Animal animal = new Animal();
        animal.write();
    }
}
```

Se va afișa „Animal" de două ori.
dog este un obiect de tipul Animal, instanțiat ca un obiect de tipul Dog (Dog este subclasă a clasei Animal, deci se poate face acest Upcast). În acest se apelează metoda write din Animal pentru dog, deoarece compilatorul știe că metoda statică write din Dog nu poate fi suprascrisă și apelează metode write din superclasă.

Dynamic binding-ul se realizează la execuție - rulare. Este utilizat în metodele supra-definite (overriding methods)

Pentru a exemplifica dynamic binding-ul putem folosi exemplul anterior, doar că definim metoda write din Dog nestatică, astfel:

```
@Override
public void write()
{ System.out.println("Dog"); }
```

Astfel la compilare, compilatorul nu va ști ce metodă să apeleze. La execuție, se va apela metoda write din subclasă (Dog).

În următorul exemplu:

```
public class Animal { int age = 10; }

public class Dog { int age = 7; }

- - - - -

Animal animal = new Dog();
System.out.println(animal.age);
```

Se va afișa 10, deoarece variabilele nu au caracter polimorfic (nu pot fi suprascrise). Astfel, se va afișa valoarea 10 (corespunzătoare tipului variabilei animal).

Subiectul 2

Care declarații sunt corecte și care nu? Argumentați răspunsurile.

```
Collection<? super Integer> col1=
    new Collection<Integer>();
```

Declarația nu este corectă, deoarece nu se poate un obiect de tip Collection (este interfață)

```
List<? extends Number> col2 = new
LinkedList<Double>();
```

Cum Double extinde Number și cum LinkedList este o clasă concretă; declarația aceasta este corectă.

```
List<? super Double> col3 = new
ArrayList<Number>();
```

Cum Number este o superclasă a lui Double și cum ArrayList este o clasă concretă, declarația este corectă.

List <? extends Double> col4 = new ArrayList <Number> ();

Cum Number nu extinde Double, declarația este greșită.

Collection <Integer> col5 = new List <Integer>();

Cum List este o interfață și cum nu se poate instanția un obiect dintr-o clasă abstractă (se pot instanția doar din subclase concrete), declarația este greșită.

Subiectul 3

a) Programul compilează. La instanțiera obiectului new Extension() se apelează constructorul din clasa Extension. Înainte de a se apela add(2), se apelează constructorul din superclasă (Base). Se apelează add(1) din Extension (metoda add este suprascrisă → dynamic binding). Deci $i += 2 \cdot 1 = 2$ (i este by default 0).

Se apelează add(2) din Extension. $i += 2 \cdot 2 = 6$.

În metoda bogo se apelează b.add(8). Cum Extension extinde Base se poate apela bogo(new Extension()).

b.add(8) → se apelează metoda add(8) din Extension (dynamic binding) =>

i += 8 · 2 = 22

Se va afișa la apelul b.print() valoarea 22.

b) Programul compilează.

La instanțierea obiectului new B() se apelează constructorul din superclasă (A). La instanțierea obiectului new C() se apelează din nou constructorul din superclasă (A).

Variabilele membre nu au caracter polimorfic în Java, deci B.this.val va fi 1, C.this.val va fi 3, iar super.val va fi 4 (în constructor clasei C s-a apelat super(4))

Deci se va afișa 1
3
4

Subiectul 4

```
public class Complex implements Comparable<Complex>
{
    public int re, im;

    public Complex () {
        this(0,0);
    }

    public Complex (int re, int im) {
        this.re = re;
        this.im = im;
    }

    public String toString () {
        return "Parte reală " + this.re +
               "Parte imaginară " + this.im + "\n";
    }

    @Override
    int compareTo ( Complex number) {
        if(number == null)
            throw new NullPointerException();

        if( this.re == number.re)
            return number.im - this.im;
        return this.re - number.re;
    }
```

```
public static void main(String args[]) {

    Vector<Complex> vect = new Vector
                              <Complex>();

    vect.add(new Complex(7, 10));
    vect.add(new Complex(3, 4));
    vect.add(new Complex(7, 4));

    vect.sort(new Comparator() {
        public int compare(Object o1,
                           Object o2) {

            Complex n1 = (Complex) o1;
            Complex n2 = (Complex) o2;
            return n1.compareTo(n2);
        }
    });

    System.out.println(vect);
}
```

Vor fi afișate [Parte reală 3 Parte imaginară 4
Parte reală 7 Parte imaginară 10
Parte reală 7 Parte imaginară 4]

Subiectul 5

Linia 1 poate să fie decomentată, întrucât tipul parametrului este schimbat și nu apar ambiguități / erori la compilare.

Linia 3 de asemenea poate fi decomentată. Metoda de pe linia 3 este suprascrisă (metoda din superclasă este implicită, iar cea din subclasă este public)

Linia 4 poate fi decomentată, fiind suprascrisă.

Linia 5 nu poate fi decomentată. O metodă supraîncărcată nu poate avea schimbat tipul argumentului returnat. Din același motiv nu poate fi decomentată nici linia 6.

Linia 7 poate fi decomentată, deoarece sunt schimbate atât tipul argumentului returnat, cât și tipul parametrului.

Linia 2 nu poate fi decomentată, deoarece metoda met suprascrie metoda met din superclasă, care nu aruncă excepția Exception.

Subiectul 6

a) Programul nu compilează, deoarece în metoda statică main se apelează metoda nestatică count într-un context static. Pentru a rezolva această problemă, putem face metoda count statică sau să modificăm main-ul astfel:

```
public static void main (String args[])
    { short myGold = 7;
      Frodo frodo = new Frodo();
      System.out.println( frodo.count(myGold,4));
    }
```

b) Programul compilează și afișează "- ic ␣ mc ␣ mf ␣ of "

În primul try este „aruncată" o excepție de tip Exception care este prinsă în catch(Exception e). Aceeași lucru se întâmplă și în cel de-al treilea try. Excepția este prinsă în Exception(ex) și s+="ic". Cum x este de tip întreg, atunci operația x/0 va genera o excepție de tip ArithmeticException care va fi prinsă în catch(Exception x) și s+="mc".

Indiferent dacă ar fi fost aruncate excepții sau nu, codul din cele două finally-uri se execută. Astfel, s+="mf␣"

și s+= "␣of".

Dacă x ar fi declarat de tip double. Operația x/0 nu ar mai fi generat nicio excepție și s-ar fi afișat „Infinity". De asemenea nu s-ar mai fi efectuat operația s+= "mc␣" și s ar fi fost în final „-ic␣mf␣of"